

# PLAYWRIGHT

## Chapitre 4 — End-to-End Web Automation Practice

*Login • Produits dynamiques • Panier • Checkout • Autocomplete • Commande • Order History*

Documentation d'apprentissage basée sur les notes du cours fournies

## À propos de ce chapitre

Ce chapitre met en pratique plusieurs concepts Playwright dans un véritable scénario End-to-End. Le parcours couvre le login, la recherche dynamique d'un produit, l'ajout au panier, le checkout, la sélection d'un pays, la création de la commande, la récupération de l'Order ID et sa recherche dans l'historique. Le scénario général est décrit dans les notes du cours. `filecite turn5file0 L4-L13`

## Sommaire

- 1. Objectifs
- 2. Scénario E2E
- 3. Login
- 4. Synchronisation après login
- 5. Recherche dynamique d'un produit
- 6. `count()`, `nth()` et boucles
- 7. Ajouter au panier
- 8. Vérifier le panier
- 9. Dropdown autocomplete
- 10. `pressSequentially()` et `delay`
- 11. Checkout et assertions
- 12. Récupérer l'Order ID
- 13. Order History
- 14. Vérifier les détails
- 15. Exemple complet
- 16. Concepts Playwright
- 17. Erreurs et bonnes pratiques
- 18. Exercices
- 19. Fiche mémo
- 20. Checklist

# 1. Objectifs

- Construire un test End-to-End réaliste.
- Rechercher un produit dynamiquement.
- Parcourir une collection de locators avec count() et nth().
- Synchroniser le test avec une interface dynamique.
- Gérer un autocomplete.
- Ajouter des assertions après les actions importantes.
- Capturer une donnée dynamique comme l'Order ID.
- Rechercher cette donnée dans une table d'historique.

# 2. Scénario E2E

1. Login
2. Sélectionner un produit et l'ajouter au panier
3. Ouvrir le panier et passer au checkout
4. Saisir les informations de paiement
5. Appliquer un coupon
6. Payer
7. Ouvrir Order History
8. Rechercher la commande
9. Afficher ses détails

**Support du cours :** Les notes détaillent surtout le login, produit, panier, checkout, pays, Order ID et Order History. Le paiement/coupon sont indiqués dans le scénario mais pas entièrement implémentés dans les extraits fournis.

# 3. Login

```
import { expect, test } from '@playwright/test';

test("Client App - E2E", async ({ page }) => {
  await page.goto("https://rahulshettyacademy.com/client");

  const email = "anshika@gmail.com";
  const password = "Iamking@0000";

  await page.locator("#userEmail").fill(email);
  await page.locator("#userPassword").fill(password);
  await page.locator("[value='Login']").click();
});
```

Le test conserve email dans une variable afin de pouvoir réutiliser cette donnée lors du checkout.

## 4. Synchronisation après login

```
await page.locator(".card-body b").first().waitFor();
```

Le premier titre de produit sert de repère : le test attend qu'un produit soit disponible avant de continuer. Cette technique est utilisée dans les exemples du support. [filecite](#)[turn5file0](#)[L29-L35](#)

```
// Autre approche présente dans l'exercice
await page.waitForLoadState("networkidle");
```

**Bonne pratique :** Attendre un élément réellement nécessaire au scénario est souvent plus ciblé qu'une attente globale du réseau.

## 5. Recherche dynamique d'un produit

Le cours montre comment chercher ADIDAS ORIGINAL sans dépendre de sa position dans la page.

```
const products = page.locator(".card-body");
const productName = "ADIDAS ORIGINAL";

const productCount = await products.count();

for (let i = 0; i < productCount; i++) {
  const product = products.nth(i);
  const currentProductName =
    await product.locator("b").textContent();

  if (currentProductName.includes(productName)) {
    await product
      .getByRole("button", { name: "Add To Cart" })
      .click();
    break;
  }
}
```

Le support utilise exactement cette logique : collection de produits, count(), boucle, nth(), lecture du nom, comparaison puis clic sur Add To Cart. [filecite](#)[turn5file0](#)[L31-L51](#)

**Idée clé :** On recherche par le contenu métier du produit plutôt que par une position fixe.

## 6. count(), nth() et boucles

| Méthode       | Rôle                                       |
|---------------|--------------------------------------------|
| count()       | Nombre d'éléments correspondant au locator |
| nth(i)        | Élément à l'index i                        |
| first()       | Premier élément                            |
| textContent() | Lire le texte                              |

```
const products = page.locator(".card-body");
const count = await products.count();

for (let i = 0; i < count; i++) {
  const product = products.nth(i);
  console.log(await product.locator("b").textContent());
}
```

**Attention :** Les index commencent à 0.

## 7. Ajouter au panier

```
await product
  .getByRole("button", { name: "Add To Cart" })
  .click();
```

Le bouton est recherché à l'intérieur du produit courant. Cela évite de cliquer sur le bouton d'un autre produit.

```
// Variante présente dans les notes
await product.locator("text= Add To Cart").click();
```

## 8. Vérifier le panier

```
await page.locator("[routerLink*='cart']").click();

await page
  .locator("h3:has-text('ADIDAS ORIGINAL')")
  .waitFor();

const visible =
  await page
    .locator("h3:has-text('ADIDAS ORIGINAL')")
    .isVisible();

expect(visible).toBeTruthy();
```

Le cours attend d'abord l'élément puis vérifie sa visibilité. [filecite](#)[turn5file0](#)[L56-L68](#)

## 9. Dropdown autocomplete

Le pays est choisi dans un autocomplete dynamique, et non dans un simple <select>. Le test saisit quelques caractères puis parcourt les suggestions.

```
await page
  .locator("[placeholder*='Country']")
  .pressSequentially("Fra", { delay: 100 });

const dropdown = page.locator(".ta-results");
await dropdown.waitFor();

const optionsCount = await dropdown.locator("button").count();

for (let i = 0; i < optionsCount; i++) {
  const option = dropdown.locator("button").nth(i);
  const currentOptionName = await option.textContent();

  if (currentOptionName.includes("France")) {
    await option.click();
    break;
  }
}
```

Cette logique est détaillée dans les notes du cours. [filecite turn5file0 L118-L140](#)

## 10. pressSequentially() et delay

Le support signale qu'une saisie trop rapide peut parfois échouer lorsque l'application ou le serveur répond lentement.

```
await page
  .locator("[placeholder*='Country']")
  .pressSequentially("ind", { delay: 150 });
```

Avec delay: 150, un délai est introduit entre chaque touche. Le support explique la séquence i → attente → n → attente → d et précise que cela laisse davantage de temps à l'application pour mettre à jour les suggestions.

[filecite turn5file0 L143-L152](#)

**À retenir :** delay est une aide de synchronisation pour ce type d'autocomplete ; ce n'est pas une règle à appliquer partout.

## 11. Checkout et assertions

```
await page.locator("text=Checkout").click();

// Choisir France dans l'autocomplete...

await expect(
  page.locator(".user__name [type='text']").first()
).toHaveText(email);
```

L'assertion vérifie que l'email affiché au checkout correspond à celui utilisé pour se connecter. Cette donnée est donc réutilisée comme référence.

```
await page.locator(".action__submit").click();

await expect(page.locator(".hero-primary"))
  .toHaveText(" Thankyou for the order. ");
```

## 12. Récupérer l'Order ID

```
const orderId =
  await page
    .locator(".em-spacer-1 .ng-star-inserted")
    .textContent();

console.log(orderId);
```

L'Order ID devient une donnée dynamique importante : elle sera utilisée pour retrouver exactement la commande dans l'historique. [filecite turn5file0 L227-L240](#)

## 13. Order History : recherche dynamique

```
await page
  .locator("button[routerlink*='myorders']")
  .click();
```

```

await page.locator("tbody").waitFor();

const orders = page.locator("tbody tr");
const ordersCount = await orders.count();

for (let i = 0; i < ordersCount; i++) {
  const currentOrderId =
    await orders.nth(i).locator("th").textContent();

  if (orderId.includes(currentOrderId)) {
    await orders
      .nth(i)
      .locator("button")
      .first()
      .click();
    break;
  }
}

```

Le support applique ici le même pattern de recherche dynamique que pour les produits : attendre, compter, parcourir, lire, comparer et agir. `filecite turn5file0 L387-L405`

**Point du cours :** Les notes indiquent d’attendre tbody avant d’utiliser count() dans cet exemple.

## 14. Vérifier les détails

```

const orderIdDetails =
  await page.locator(".col-text").textContent();

expect(orderId.includes(orderIdDetails))
  .toBeTruthy();

```

L’assertion finale vérifie que l’Order ID des détails correspond à celui capturé après le paiement.

`filecite turn5file0 L406-L410`

## 15. Exemple complet commenté

```

import { expect, test } from '@playwright/test';

test("Client App - Complete E2E Order Flow", async ({ page }) => {
  const email = "anshika@gmail.com";
  const password = "Iamking@0000";
  const productName = "ADIDAS ORIGINAL";

  // 1. Login
  await page.goto("https://rahulshettyacademy.com/client");
  await page.locator("#userEmail").fill(email);
  await page.locator("#userPassword").fill(password);
  await page.locator("[value='Login']").click();

  // 2. Wait for products
  await page.locator(".card-body b").first().waitFor();

  // 3. Find product dynamically
  const products = page.locator(".card-body");

```

```

const productCount = await products.count();

for (let i = 0; i < productCount; i++) {
  const product = products.nth(i);
  const name = await product.locator("b").textContent();

  if (name.includes(productName)) {
    await product
      .getByRole("button", { name: "Add To Cart" })
      .click();
    break;
  }
}

// 4. Cart
await page.locator("[routerLink*='cart']").click();
const cartProduct = page.locator(
  `h3:has-text('${productName}')`
);
await cartProduct.waitFor();
expect(await cartProduct.isVisible()).toBeTruthy();

// 5. Checkout
await page.locator("text=Checkout").click();

// 6. Country autocomplete
await page
  .locator("[placeholder*='Country']")
  .pressSequentially("Fra", { delay: 100 });

const dropdown = page.locator(".ta-results");
await dropdown.waitFor();

const optionsCount =
  await dropdown.locator("button").count();

for (let i = 0; i < optionsCount; i++) {
  const option = dropdown.locator("button").nth(i);
  const name = await option.textContent();

  if (name.includes("France")) {
    await option.click();
    break;
  }
}

// 7. Validate checkout data
await expect(
  page.locator(".user__name [type='text']").first()
).toHaveText(email);

// 8. Place order
await page.locator(".action__submit").click();
await expect(page.locator(".hero-primary"))
  .toHaveText(" Thankyou for the order. ");

// 9. Capture Order ID

```



```

const orderId =
  await page
    .locator(".em-spacer-1 .ng-star-inserted")
    .textContent();

console.log("Order ID:", orderId);

// 10. Open Order History
await page
  .locator("button[routerlink*='myorders']")
  .click();

await page.locator("tbody").waitFor();

// 11. Find the order dynamically
const orders = page.locator("tbody tr");
const count = await orders.count();

for (let i = 0; i < count; i++) {
  const currentId =
    await orders.nth(i).locator("th").textContent();

  if (orderId.includes(currentId)) {
    await orders
      .nth(i)
      .locator("button")
      .first()
      .click();
    break;
  }
}

// 12. Validate details
const details =
  await page.locator(".col-text").textContent();

expect(orderId.includes(details)).toBeTruthy();
});

```

## 16. Concepts Playwright mobilisés

| Concept                    | Exemple du chapitre              | Rôle                          |
|----------------------------|----------------------------------|-------------------------------|
| <b>Locator</b>             | #userEmail, .card-body, tbody tr | Identifier des éléments       |
| <b>waitFor()</b>           | .card-body b, .ta-results, tbody | Synchroniser                  |
| <b>count()</b>             | Produits, options, commandes     | Compter une collection        |
| <b>nth()</b>               | Produit/option/ligne courant     | Accéder à un élément          |
| <b>textContent()</b>       | Nom, pays, Order ID              | Lire du texte                 |
| <b>isVisible()</b>         | Produit dans le panier           | Lire un état                  |
| <b>expect()</b>            | Checkout, confirmation, détails  | Valider                       |
| <b>getByRole()</b>         | Add To Cart                      | Locator orienté accessibilité |
| <b>pressSequentially()</b> | Country                          | Saisie progressive            |

## 16.1 Pattern réutilisable

```
const items = page.locator("...");

await items.first().waitFor();

const count = await items.count();

for (let i = 0; i < count; i++) {
  const item = items.nth(i);
  const text = await item.locator("...").textContent();

  if (text.includes("valeur recherchée")) {
    await item.locator("button").click();
    break;
  }
}
```

## 17. Erreurs fréquentes et bonnes pratiques

**Produit à position fixe** : Chercher par nom plutôt que supposer nth(0), nth(1), etc.

**Mauvais bouton Add To Cart** : Rechercher le bouton à l'intérieur du produit courant.

**Attentes arbitraires** : Préférer une attente liée à un élément ou à un état utile.

**Autocomplete traité comme un select** : Saisir dans l'input, attendre les suggestions puis les parcourir.

**Saisie trop rapide** : Utiliser pressSequentially avec delay lorsque le contexte l'exige.

**Pas d'assertion** : Valider les résultats importants après les actions.

**Order ID perdu** : Conserver l'identifiant dans une variable et le réutiliser.

**Order History non synchronisé** : Attendre tbody avant de parcourir les lignes.

## 18. Exercices

1. Remplacer ADIDAS ORIGINAL par ZARA COAT 3.
2. Afficher tous les titres avec allTextContents().
3. Ajouter une assertion confirmant la présence du produit dans le panier.
4. Tester un autre pays dans l'autocomplete.
5. Comparer plusieurs valeurs de delay.
6. Vérifier que l'email du checkout est identique à celui du login.
7. Capturer et afficher l'Order ID.
8. Rechercher la commande dynamiquement dans Order History.
9. Ouvrir les détails de la bonne commande.

10. Ajouter une assertion finale sur l'Order ID.

## 19. Fiche mémo

| Objectif        | Syntaxe                                                            |
|-----------------|--------------------------------------------------------------------|
| Naviguer        | <code>await page.goto(url)</code>                                  |
| Remplir         | <code>await locator.fill(value)</code>                             |
| Cliquer         | <code>await locator.click()</code>                                 |
| Attendre        | <code>await locator.waitFor()</code>                               |
| Compter         | <code>await locator.count()</code>                                 |
| Élément i       | <code>locator.nth(i)</code>                                        |
| Premier         | <code>locator.first()</code>                                       |
| Texte           | <code>await locator.textContent()</code>                           |
| Tous les textes | <code>await locator.allTextContents()</code>                       |
| Visible         | <code>await locator.isVisible()</code>                             |
| Assertion vraie | <code>expect(value).toBeTruthy()</code>                            |
| Assertion texte | <code>await expect(locator).toHaveText(text)</code>                |
| Autocomplete    | <code>await locator.pressSequentially(text, { delay: 150 })</code> |
| Par rôle        | <code>locator.getByRole('button', { name: 'Add To Cart' })</code>  |
| Pause debug     | <code>await page.pause()</code>                                    |

## 20. Checklist de validation

- ☐ Je sais construire un workflow E2E.
- ☐ Je sais attendre le chargement des produits.
- ☐ Je comprends `count()` et `nth()`.
- ☐ Je sais parcourir une liste avec `for`.
- ☐ Je sais rechercher un produit par son nom.
- ☐ Je sais limiter un bouton au produit courant.
- ☐ Je sais vérifier le panier.
- ☐ Je comprends un autocomplete dynamique.
- ☐ Je sais utiliser `pressSequentially()`.
- ☐ Je comprends `delay`.
- ☐ Je sais conserver une donnée dynamique.
- ☐ Je sais récupérer un Order ID.
- ☐ Je sais parcourir une table.
- ☐ Je sais retrouver une ligne par son identifiant.
- ☐ Je sais vérifier les détails de la commande.

## Conclusion

Ce chapitre marque le passage vers une vraie automatisation End-to-End. Le test ne vérifie plus une action isolée : il transporte des données d'une étape à l'autre, recherche dynamiquement des éléments et valide le résultat final. Les deux patterns les plus importants à retenir sont la recherche dynamique dans une collection et la synchronisation avec des interfaces dynamiques.

```
Login
↓
Wait for products
↓
Find product dynamically
↓
Add to cart
↓
Verify cart
↓
Checkout
↓
Autocomplete country
↓
Validate checkout
↓
Place order
↓
Capture Order ID
↓
Order History
↓
Find order dynamically
↓
Verify details
```